

Algoritmalar ve Veri Yapıları Bootcamp Ödev 1.

1. Verilen iki tam sayının en büyük ortak bölenini (EBOB) bulmak için çeşitli algoritmalar öne sürülmüştür. Bunlardan belki en eski olanı Euclid'in algoritmasıdır ve aşağıda verilen şu eşitliğin:

$$EBOB(m,n) = EBOB(n, m \bmod n)$$

ikinci değer ($m \bmod n$) sıfıra ulaşıncaya kadar tekrar edilmesine dayanır. İkinci değer sıfır olduğu anda ilk değer aranan EBOB değeridir (burada ' $\bmod$ ' işlemi m 'nin n 'ye bölme işlemi sonucunda ortaya çıkan kalan değerini işaret etmektedir).

İki tam sayının EBOB'unu bulan diğer bir algoritmaysa '*ardışık tam sayı kontrol etme algoritması*' olarak adlandırılır ve şu ana fikre dayanmaktadır: *İki sayının en büyük ortak böleni (EBOB'u) bu iki sayının küçük olanından daha büyük bir sayı olamaz.* Bu argümana dayanarak, algoritma verilen iki sayının küçük olanından başlayıp aşağıya 1 sayısına doğru birer birer azalarak ilerler. Bu yolda bulunan ilk ortak bölen EBOB'dur.

Yukarıda bahsedilen iki algoritmayı (*Euclid'in algoritması* ve *ardışık tam sayı kontrol etme algoritması*) derslerde gördüğümüz pseudocode formatında yazınız ve $m=31415$ ve $n=14142$ girdi sayıları için, her iki algoritmanın temel operasyonlarının toplam çalıştırılma/koşturulma sayılarını birbirleriyle karşılaştırınız.

2. Size n elemanlı bir tamsayı dizisi A ve bir de tamsayı k veriliyor. Verilen dizinin ardışık indekslerine yerleştirilen ve toplamı k 'ya eşit olan eleman serisini bulan algoritmayı pseudocode olarak yazınız. Algoritmanız, eğer A 'da toplamı k 'ye eşit olan böyle bir seri varsa serinin başladığı indeksin değerini, A 'da toplamı k 'ye eşit olan böyle bir seri yoksa da -1 döndürmelidir. Algoritmanızın **en kötü** ve **en iyi** durumlarda temel operasyonlarının kaçar kez çalıştırıldığına dair ifadeleri n cinsinden ayrı ayrı belirtiniz.

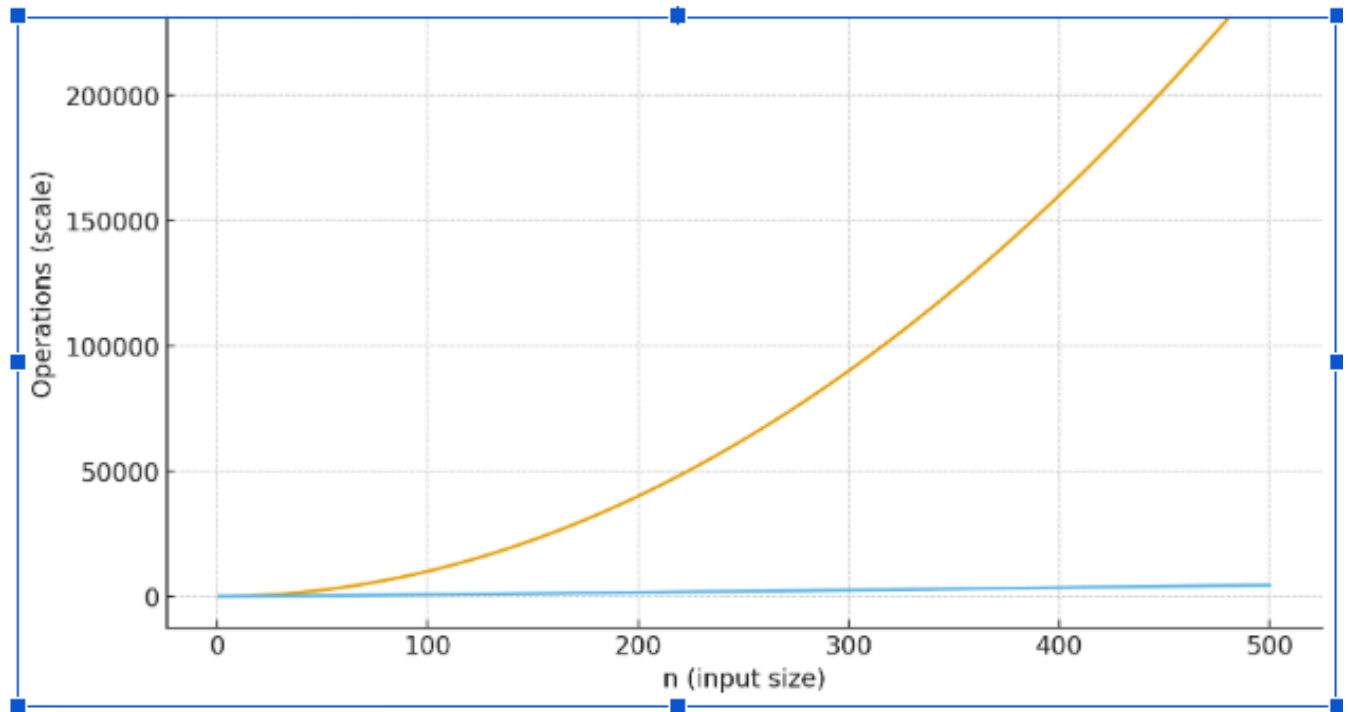
3. Bu ödevde, iyi bilinen iki sıralama algoritmasının, selection-sort ve bubble-sort algoritmalarının zaman karmaşıklığını deneysel olarak, yani bunları uygulayarak ve çeşitli deneyler yaparak karşılaştırmanız beklenmektedir.

Öncelikle her iki algoritmayı da Java programlama diliyle uygulamanız gerekmektedir. Ardından uygulamalarınızı birkaç kez çalıştırın ve aşağıdaki tabloyu her iki algoritma için ayrı ayrı doldurun:

	Selection Sort	Bubble Sort
--	----------------	-------------

Problem boyutu	Temel işlemin kaç kez gerçekleştirildiği	Milisaniye cinsinden koşu zamanı	Temel işlemin kaç kez gerçekleştirildiği	Milisaniye cinsinden koşu zamanı
10				
50				
100				
500				
1000				
5000				
10000				
50000				
100000				
200000				
300000				
400000				
500000				
1000000				

Ardından, elde ettiğiniz sonuçları iki boyutlu bir koordinat sisteminde birkaç grafik aracılığıyla gösterin; burada bir boyut problemin boyutunu, diğer boyut ise bir grafik için "temel işlemin kaç kez gerçekleştirildiğini", diğer grafik için ise uygulamanızın verilen değerleri sıralamak için ihtiyaç duyduğu "çalışma süresini" (milisaniye cinsinden) temsil etmelidir. Yani, iki sıralama algoritmasını temel işlemlerinin gerçekleştirilme sayılarına göre karşılaştıran bir grafik ve iki sıralama algoritmasını çalışma sürelerine göre karşılaştıran başka bir grafik oluşturmalsınız. Aşağıdaki biçimde iki farklı grafik oluşturmalsınız; birinde dikey eksen "temel işlemin kaç kez gerçekleştirildiğini", diğerinde ise uygulamanın "çalışma süresini (milisaniye cinsinden)" göstermelisiniz.



Algoritmalarınızın çalışma sürelerini hesaplamak için aşağıdaki sınıfı kronometre olarak kullanabilirsiniz.

```
public class Stopwatch {  
    private final long  
        start;  
  
    /** * Initializes a new stopwatch. */  
    public Stopwatch() {  
        start = System.currentTimeMillis();  
    }  
  
    /** * Returns the elapsed CPU time (in seconds) since the stopwatch was  
        created.  
    * * @return elapsed CPU time (in seconds) since the stopwatch was created  
    */  
  
    public double elapsedTime() {  
        long now = System.currentTimeMillis();  
        return (now - start);  
    }  
}
```

}

Aşağıdaki kod örneği, yukarıda verilen kronometreyi kullanarak algoritmalarınızın çalışma süresini nasıl ölçebileceğinizi göstermektedir.

```
public static void main(String[] args)
{
    In in = new In(args[0]);
    int[] a = in.readAllInts();
    Stopwatch stopwatch = new Stopwatch();
    Algoritmanızı burada çalıştırın
    double time = stopwatch.elapsedTime();
    StdOut.println("elapsed time " + time);
}
```